

Betriebssysteme

Musterlösungen zu den Probeklausuren

Name:

Matrikelnummer:

Hinweis: Diese Musterlösungen sind im Klausurblatt-Stil aufgebaut: Aufgabenstruktur wie im Blatt, Lösung direkt darunter. Zeitdiagramme und Graphen sind als einzutragende Balken/Kanten angegeben, damit du sie 1:1 in die Raster übertragen kannst.

Inhalt

1. Klausur Betriebssysteme SS 2007
2. Klausur Betriebssysteme SS 2008
3. Klausur Betriebssysteme SS 2009
4. Klausur Betriebssysteme SS 2011
5. Prüfung Betriebssysteme WS 2015/16
6. Prüfung Betriebssysteme WS 2019/20
7. Exam Operating Systems SS 2021

Wichtig: Bei RMS-Aufgaben unterscheide ich zwischen notwendiger CPU-Auslastung $U \leq 1$ und der hinreichenden RMS-Garantiegrenze. Liegt U dazwischen, ist die einfache Vorlesungsgrenze nicht beweisend; ein genauer Antwortzeittest kann im Einzelfall trotzdem funktionieren.

Klausur Betriebssysteme SS 2007 - Musterlösung

Aufgabe 1: E/A, DMA

a) Drei Formen der E/A-Organisation: programmierte E/A/Polling, interruptgesteuerte E/A, DMA.

b) Memory-mapped I/O: Gerätereister liegen im normalen Adressraum und werden mit normalen Lade-/Speicherbefehlen angesprochen. Vorteil: einheitliche Befehle/Adressierung; Nachteil: Teile des Adressraums gehen für Geräte verloren, Caching/Schutz muss beachtet werden. Port-mapped I/O: eigener I/O-Adressraum und spezielle I/O-Befehle. Vorteil: Trennung von Speicher und Geräten; Nachteil: eigene Befehle, weniger einheitlich.

c) DMA = Direct Memory Access. Die CPU initialisiert einen Controller, der danach einen Datenblock direkt zwischen Gerät und Hauptspeicher überträgt. Die CPU muss nicht jedes Byte selbst kopieren; am Ende meldet der Controller den Abschluss meist per Interrupt.

d) Mindestens: Geräte-/Plattenadresse bzw. Blocknummer, Zieladresse im Hauptspeicher, Länge/Anzahl Bytes oder Blöcke, Richtung Leseauftrag, ggf. Steuer-/Statusinformationen.

e) Cycle Stealing bedeutet, dass der DMA-Controller einzelne Speicherbuszyklen benutzt und die CPU in diesen Zyklen kurz ausbremst.

Aufgabe 2: RMS

a) Rate Monotonic heißt so, weil kürzere Periodendauer eine höhere Aktivierungsrate bedeutet. Je höher die Rate, desto höher die feste Priorität.

b) RMS ist unter den klassischen Annahmen optimal unter allen Verfahren mit festen Prioritäten. Wenn irgendein festes Prioritätsverfahren eine periodische Taskmenge schedulen kann, dann schafft RMS sie ebenfalls. Das heißt nicht, dass RMS jede beliebige Taskmenge schafft.

c) Zeitdiagramm:

A: 0-2, 3,5-5,5, 16-20 | B: 2-3,5, 12-13,5 | C: 5,5-7,5, 10-11 | D: 7,5-10
Verdrängungen: $\forall$ bei $t=2$ und $\forall$ bei $t=7,5$.

d) Ja, alle bis dahin relevanten Deadlines werden eingehalten: A0 fertig $5,5 \leq 16$; B2 fertig $3,5 \leq 12$; C3 fertig $11 \leq 27$; D7,5 fertig $10 \leq 27,5$; B12 fertig $13,5 \leq 22$. A16 hat Deadline 32 und ist bei $t=20$ noch nicht fällig.

e) Ja. $U = 4/16 + 1,5/10 + 3/24 + 2,5/20 = 0,25 + 0,15 + 0,125 + 0,125 = 0,65 \leq 1$. Die benötigte CPU-Zeit passt also auf eine einzelne CPU.

f) Ja, mit der RMS-Grenze: Für $n=4$ gilt $4(2^{(1/4)}-1) \approx 0,757$. Da $U=0,65$ darunter liegt, ist Deadline-Einhaltung unter den RMS-Annahmen garantiert.

Aufgabe 3: Semaphore/Deadlock

	B=true	B=false
A=true	Nein. P1 kann ggf. S3 bekommen; P2 hält S3 nicht dauerhaft, bevor es S1/S2 nimmt.	Ja. Möglich: P1 hält S2 und wartet auf S3; P2 hält S3 und wartet auf S2.
A=false	Nein. P2 gibt S3 frei, bevor eine echte Kreiswartebeziehung entstehen kann.	Nein. Wartesituationen lösen sich auf, weil jeweils eine benötigte Semaphore frei werden kann.

b). Schlecht strukturiert: verschachtelte und verzweigte Sperrlogik, unterschiedliche Reihenfolgen, Signale innerhalb der Zweige. Genau solche Programme sind schwer zu prüfen und deadlockanfällig.

c). `wait(S)` ist atomar. Ist S frei/positiv, wird S belegt/dekrementiert und der Prozess läuft weiter. Ist S nicht verfügbar, wird der Prozess blockiert und in die Warteschlange der Semaphore eingetragen. `signal(S)` gibt S frei/inkrementiert; wartet ein Prozess, wird einer deblockiert und in den Ready-Zustand versetzt.

Aufgabe 4: Paging

a). Bei 32 Bit und Rahmengröße 2^k : $2^{(32-k)}$ Seitentableneinträge. Falls mit 2 KiB gemeint ist: $2^{(32-11)}=2^{21}$.

b). Mehrstufige Seitentabellen, invertierte Seitentabelle oder Seitentabellen nur für tatsächlich benutzte Bereiche.

c). Bei 2 KiB Rahmen: Offset der Startadresse $0x780 = 1920$. $\text{floor}((1920 + 4225 - 1)/2048)+1 = 4$. Das Array benötigt also 4 Rahmen.

d). Typische Bits: Present/Valid-Bit: Seite im Hauptspeicher? Referenced/Use-Bit: wurde seit letzter Prüfung benutzt, wichtig für Clock/LRU-Näherung. Dirty/Modified-Bit: wurde verändert, muss beim Ersetzen zurückgeschrieben werden.

Aufgabe 5: Seitenersetzung

Seq	7	5	8	6	5	9	6	9	8	6	5	6
OPT R1	7	7	7	6	6	6	6	6	6	6	6	6
OPT R2	-	5	5	5	5	9	9	9	9	9	5	5
OPT R3	-	-	8	8	8	8	8	8	8	8	8	8
F	F	F	F	F		F					F	

Seq	7	5	8	6	5	9	6	9	8	6	5	6
LRU R1	7	7	7	6	6	6	6	6	6	6	6	6
LRU R2	-	5	5	5	5	5	5	5	8	8	8	8
LRU R3	-	-	8	8	8	9	9	9	9	9	5	5
F	F	F	F	F		F			F		F	

Second Chance: gleiche Seitenfolge wie LRU in dieser Aufgabe: Faults bei 7, 5, 8, 6, 9, 8, 5. Zustände nach jedem Zugriff: (7,-,-), (7,5,-), (7,5,8), (6,5,8), (6,5,8), (6,5,9), (6,5,9), (6,5,9), (6,8,9), (6,8,9), (6,8,9), (6,8,5), (6,8,5). Use-Bits nach Clock-Regel aktualisieren.

Klausur Betriebssysteme SS 2008 - Musterlösung

Aufgabe 1: Threads/Prozesswechsel

a) Threads sind leichter als Prozesse: schnelleres Erzeugen/Wechseln, gemeinsamer Adressraum, einfache Kommunikation. Nachteile: weniger Isolation; Fehler/Race Conditions betreffen denselben Prozess; Synchronisation ist nötig.

b) Benutzerthreads: sehr schnell im User Space, aber blockierender Systemaufruf kann den ganzen Prozess blockieren und echte Mehrkernparallelität ist eingeschränkt. Kernelthreads: vom Kernel separat planbar, Blockierung einzelner Threads und echte Parallelität möglich; dafür mehr Overhead.

c) Jacketing wird eingesetzt, damit blockierende Systemaufrufe bei Benutzerthreads nicht den gesamten Prozess blockieren. Eine Bibliothek erkennt/umhüllt blockierende Aufrufe und ersetzt sie z.B. durch nichtblockierende Varianten oder organisiert den Aufruf über Hilfsmechanismen.

d) Beim Prozesswechsel $P1 \rightarrow P2$: Zustand von $P1$ sichern (PC, Register, Stackpointer, Statuswort, ggf. Speicherverwaltungsinfos) im PCB; Zustand/Priorität/Zustandstabellen aktualisieren; PCB von $P2$ auswählen; Adressraum/Speicherabbildung ggf. umschalten; Register/PC/SP von $P2$ laden; $P2$ in aktiv/laufend setzen; Sprung zur gespeicherten Fortsetzungsadresse.

e) Die Anwendung muss parallelisierbare, unabhängige bzw. nebenläufig ausführbare Teile haben und mehrere lauffähige Threads/Prozesse erzeugen.

Aufgabe 2: EDF + RMS

a/b) Zeitdiagramm:

T1: 0-5, 12-14 | T3: 5-9 | T4: 9-12 | T2: 14-20

Verdrängung: v bei $t=5$, weil T3 mit absoluter Deadline 10 früher ist als T1 mit Deadline 13.

c) Nicht alle Deadlines werden eingehalten. T1 hat absolute Deadline 13 und wird erst bei $t=14$ fertig. T2 wird bei 20 fertig, T3 bei 9, T4 bei 12.

d) RMS-Priorität nach relativer Deadline/Periode: $T4(P=4) > T3(P=5) > T1(P=13) > T2(P=18)$.

e) Für T1/T2: $U = 7/13 + 6/18 \approx 0,872$. Notwendig ist $U \leq 1$, also ist es nicht grundsätzlich unmöglich. Die einfache RMS-Garantie für $n=2$ lautet $2(\sqrt{2}-1) \approx 0,828$; diese wird überschritten. Mit der reinen Vorlesungsgrenze ist also keine Immer-Garantie ableitbar; genauer geprüft ist die konkrete Zweitaskmenge aber schedulbar ($R1=7 \leq 13$, $R2=13 \leq 18$).

Aufgabe 3: Race Conditions

a) Atomare Statements, Reihenfolge je Thread bleibt erhalten. Minimum: 8 bei $S21 \rightarrow S22 \rightarrow S11 \rightarrow S12$. Maximum: 24 bei $S21 \rightarrow S11 \rightarrow S12 \rightarrow S22$.

b) Wenn einzelne Statements unterbrochen werden können, wird die Ergebnismenge größer. Zerlegt man $x=x+1$ und $x=3*x$ in Lesen/Berechnen/Schreiben, kann z.B. Minimum 6 entstehen: S11, S21, S12 liest 5, S22 liest 5, S22 schreibt 15, S12 schreibt 6. Maximum bleibt 24.

c) Schutz durch kritische Abschnitte mit Mutex/Semaphor/Monitor, atomare Test-and-Set-Operationen bzw. Locks, oder strukturierte Synchronisation/Message Passing.

Aufgabe 4: Paging

a) $4 \text{ KiB} = 2^{12}$, also Offset = untere 12 Bit = $0x078$; Seitennummer = obere 20 Bit = $0x7A5B3$.

b) Kleinste Adresse derselben Seite: $0x7A5B3000$. Größte: $0x7A5B3FFF$.

c) Anzahl Seitentableneinträge: 2^{20} .

d) Skizze: virtuelle Adresse $\rightarrow$ Zerlegung in Seitennummer und Offset; Seitennummer indexiert Seitentabelle; Eintrag liefert Rahmennummer plus Statusbits; physische Adresse = Rahmennummer || unveränderter Offset.

e) Insgesamt 2^{20} virtuelle Seiten. Bei gleich großen Haupt- und Untertabellen: 2^{10} Einträge in der Haupttabelle und 2^{10} Einträge je Unterseitentabelle.

Klausur Betriebssysteme SS 2009 - Musterlösung

Aufgabe 1: Scheduling-Theorie

a) Ziele: hohe CPU-Auslastung, hoher Durchsatz, kurze Antwortzeit, kurze Wartezeit, kurze Durchlaufzeit, Fairness/kein Verhungern, Deadline-Einhaltung, geringer Overhead.

b) Zielkonflikt: kurze Antwortzeit durch häufige Verdrängung erhöht Kontextwechsel-Overhead und kann den Durchsatz senken.

c) Verdrängung bedeutet, dass einem laufenden Prozess die CPU entzogen wird, obwohl er selbst nicht blockiert oder fertig ist.

d) Verdrängung kann stattfinden bei Ablauf eines Quantums, Ankunft eines höherpriorären bzw. dringlicheren Tasks, Ablauf/Änderung von Prioritäten, Echtzeitereignis/Deadline-relevanter Ankunft.

e) FCFS: erste Ankunft zuerst, nicht präemptiv. SJF: kürzeste erwartete Rechenzeit zuerst. SRT: kürzeste Restzeit, präemptiv. Round Robin: Zeitscheiben mit zyklischer Queue. EDF: früheste absolute Deadline zuerst. RMS: kürzeste Periode hat höchste feste Priorität.

Aufgabe 2: Semaphore/Deadlocks

a) Ja, der kritische Abschnitt ist geschützt. P1/P2 teilen S1, P1/P3 teilen S2, P2/P3 teilen S3. Damit kann kein Paar gleichzeitig im kritischen Abschnitt sein.

b) P2 im kritischen Abschnitt hält S1 und S3. P1 wartet vor dem kritischen Abschnitt auf S1. P3 kann S2 nehmen und wartet dann auf S3. Graph: S1 -> P2, S3 -> P2, S2 -> P3, P1 -> S1, P3 -> S3.

c) Kein Deadlock, weil P2 nicht wartet und den kritischen Abschnitt verlassen kann. Danach werden S1/S3 frei.

d) Nein, bei beliebigem Ablauf entsteht kein Deadlock. Die Sperren werden konsistent in einer globalen Ordnung genommen: S1 vor S2/S3, S2 vor S3. Eine zirkuläre Wartebedingung kann dadurch nicht entstehen.

e) Die Implementierung ist ungeeignet, weil Test und Setzen nicht atomar sind. Zwei Prozesse können beide `my_sems[sem]==0` sehen, beide die Schleife verlassen, beide auf 1 setzen und gleichzeitig eintreten. Außerdem ist Busy Waiting ineffizient.

f) Methoden: Interrupts sperren, atomare Test-and-Set/Compare-and-Swap-Operationen, Mutex/Semaphore, Monitore, Peterson-Algorithmus, Message Passing.

Aufgabe 3: Speicher

a) 256 MiB / 64 KiB = $2^{28} / 2^{16} = 2^{12}$ Rahmen.

b) 32 Bit / 64 KiB Seiten: $2^{32} / 2^{16} = 2^{16}$ Seitentabelleneinträge.

c) 340 KiB / 64 KiB = 5,3125, also minimal 6 Rahmen. Bei ungünstigem Startoffset maximal 7 Rahmen.

d) Adresse 0x470A78BB: Seite 0x470A, Offset 0x78BB. Tabelle liefert Rahmen 0x1B3. Physische Adresse: 0x01B378BB.

e) Physische Adresse 0x31A9175: Rahmen 0x31A, Offset 0x9175. In der Tabelle gehört Rahmen 0x31A zu Seite 0x4708. Virtuelle Adresse: 0x47089175.

f) 0x4709FFFF liegt auf Seite 0x4709, P-Bit 0. Es entsteht ein Page Fault: Trap in den Kernel, Prozess blockieren, freie Kachel oder Opfer suchen, ggf. Dirty-Seite zurückschreiben, Seite laden, Seitentabelle/TLB aktualisieren, Instruktion wiederholen.

Aufgabe 4: Thrashing und Ersetzung

a) Thrashing entsteht, wenn die Working Sets der Prozesse nicht in den Hauptspeicher passen. Das System verbringt dann mehr Zeit mit Page Faults und Auslagern als mit Nutzarbeit.

b) Lokalität hilft gegen Thrashing: zeitliche Lokalität nutzt kürzlich verwendete Seiten erneut; räumliche Lokalität nutzt benachbarte Seiten. Dadurch sinkt die benötigte aktive Seitenmenge.

Seq	8	7	8	3	2	8	3	7	3
OPT R1	8	8	8	8	8	8	8	7	7
OPT R2	4	7	7	3	3	3	3	3	3
OPT R3	2	2	2	2	2	2	2	2	2
F		F		F				F	

d) Clock: Faults bei 7, 3, 2, 8, 7, 3. Zustände nach jedem Zugriff: (8,4,2), (8,7,2), (8,7,2), (8,7,3), (2,7,3), (2,8,3), (2,8,3), (2,8,7), (3,8,7). Use-Bits und Zeiger werden nach Second-Chance-Regel aktualisiert.

Klausur Betriebssysteme SS 2011 - Musterlösung

Aufgabe 1: Dateisysteme/Abstraktion

a) Beispiele: Prozess abstrahiert CPU-Ausführung; Datei abstrahiert Sekundärspeicher; virtueller Speicher abstrahiert Hauptspeicher; Socket abstrahiert Netzwerkkommunikation; Gerätedateien abstrahieren E/A-Geräte.

b) Sequentieller Zugriff: Lesen/Schreiben der Reihe nach. Direkter/wahlfreier Zugriff: Zugriff auf beliebige Positionen. Indexierter Zugriff: Zugriff über Index-/Metadatenstruktur.

c) Verkettete Listen sind schlecht für Auslagerungsdateien, weil wahlfreier Zugriff auf eine Seite lange Kettenverfolgung erfordert. Virtueller Speicher braucht schnellen direkten Zugriff auf Seiten.

d) Block = 1 KiB, ein indirekter Block enthält 256 Adressen. Datenblöcke: $8 + 8 \cdot 256 = 2056$. Max. Dateigröße: $2056 \text{ KiB} = 2.105.344 \text{ Byte}$.

Aufgabe 2: Scheduling

a) Pro CPU kann nur ein Prozess gleichzeitig aktiv/laufend sein. Blockiert sein können theoretisch beliebig viele Prozesse, begrenzt nur durch Speicher/Verwaltungsressourcen.

b) Round Robin mit Quantum 3 und 1 Zeiteinheit Wechselkosten: P1 0-3, P2 4-7, P3 8-11, P4 12-15, P1 16-19, P2 ab 20.

c) Scheduler und Dispatcher benötigen zusammen 1 Zeiteinheit pro Prozesswechsel.

d/e) EDF:

T4: 0-1, 4-9 | T1: 1-4 | T3: 9-13 | T2: 13-16

Verdrängung: v bei $t=1$, weil T1 Deadline 7 früher ist als T4 Deadline 10.

f) Nicht alle Deadlines werden eingehalten: T3 hat Deadline 12, wird aber erst bei $t=13$ fertig. T1, T4 und T2 werden rechtzeitig fertig.

g) Für T1/T2 periodisch: $U = 3/7 + 3/17 \approx 0,605$. Für $n=2$ ist die RMS-Grenze $0,828$. Da $0,605 < 0,828$, garantiert RMS die Deadline-Einhaltung.

Aufgabe 3: Speicher

a) $1024 \text{ MiB} / 32 \text{ KiB} = 2^{30} / 2^{15} = 2^{15}$ Rahmen.

b) $2^{32} / 2^{15} = 2^{17}$ Seitentabelleneinträge.

c) Es gibt mehr virtuelle Seiten als physische Rahmen, weil der virtuelle Adressraum 4 GiB umfasst, der physische Speicher aber nur 1 GiB.

d) 32 KiB = 0x8000. Adresse 0x0F3DA7A1 liegt in der Seite 0x0F3D8000 bis 0x0F3DFFFF.

e) Mit 32-Bit-Adressierung maximal 4 GiB direkt adressierbarer Hauptspeicher. Der virtuelle Adressraum eines einzelnen 32-Bit-Prozesses ist ebenfalls auf 4 GiB begrenzt; insgesamt können mehrere Prozesse durch getrennte Adressräume/backing store mehr virtuelle Speicherbereiche haben.

Aufgabe 4: Seitenersetzung

Seq	7	5	8	6	5	9	6	9	8	6	9	6
OPT R1	7	7	7	6	6	6	6	6	6	6	6	6
OPT R2	-	5	5	5	5	9	9	9	9	9	9	9
OPT R3	-	-	8	8	8	8	8	8	8	8	8	8
F	F	F	F	F		F						

b) Optimalstrategie heißt so, weil sie die Seite ersetzt, deren nächster Zugriff am weitesten in der Zukunft liegt.

c) Das Betriebssystem kann OPT nicht direkt verwenden, weil es die zukünftigen Zugriffe nicht kennt. Es dient als theoretische Untergrenze/Vergleichsmaßstab.

d) Clock: Faults bei 7, 5, 8, 6, 9, 8. Zustände: (7,-,-), (7,5,-), (7,5,8), (6,5,8), (6,5,8), (6,5,9), (6,5,9), (6,5,9), (6,8,9), (6,8,9), (6,8,9), (6,8,9).

Aufgabe 5: Deadlocks

a) Kanten: B2→P1, P1→B5; B6→P2, P2→B2 und P2→B3; B3→P3, P3→B1; B1→P4, P4→B6; B5→P5, P5→B4.

b) Ja. Es gibt den Zyklus P2 -> B3 -> P3 -> B1 -> P4 -> B6 -> P2.

c) Es genügt, einen Prozess aus dem Zyklus zu terminieren: P2 oder P3 oder P4.

d) Zyklusanalyse ist zur Verhinderung allgemein unpraktikabel, weil sie laufend globale Zustände prüfen müsste, bei mehreren Instanzen komplizierter ist und Deadlocks erst nahe am Eintritt erkennt. Verhinderung arbeitet besser über feste Regeln.

e) Methoden: globale Ressourcenordnung, alle Ressourcen auf einmal anfordern, Hold-and-Wait verhindern, Ressourcen entziehen/Preemption, Spooling/Virtualisierung exklusiver Ressourcen.

Prüfung Betriebssysteme WS 2015/16 - Musterlösung

Aufgabe 1: Scheduling

a) Mindestens sechs: CPU-Auslastung, Durchsatz, Antwortzeit, Wartezeit, Durchlaufzeit, Fairness, Deadline-Einhaltung, geringer Overhead, Prioritäten beachten.

b) Beispiel: kurze Antwortzeit vs. hoher Durchsatz. Häufige Verdrängung verbessert Interaktivität, erhöht aber Kontextwechselkosten.

c) Verdrängung: laufender Prozess verliert CPU, obwohl er nicht fertig/blockiert ist. Sein Zustand wird gesichert; er geht zurück in Ready. Ein anderer Prozess wird geladen und läuft.

d) Präemptives Scheduling erlaubt genau diese zwangsweise Unterbrechung laufender Prozesse.

Aufgabe 2: EDF/RMS

a) EDF:

T4: 0-3, 9-13 | T1: 3-5, 7-9 | T2: 5-7 | T3: 13-15

Verdrängungen: v bei $t=3$ und v bei $t=5$. Deadline-Miss: x bei $t=12$ für T4.

b) RMS-Prioritäten: $T2(P=5) > T1(P=8) > T4(P=12) > T3(P=20)$.

c) Nur T1/T2: $U=4/8+2/5=0,9$. Notwendiges Kriterium $U \leq 1$ erfüllt. RMS-Garantiegrenze für $n=2$ ist 0,828; 0,9 liegt darüber, also keine Garantie durch die einfache Grenze.

Aufgabe 3: Speicher

a) $2048 \text{ MiB} / 64 \text{ KiB} = 2^{31} / 2^{16} = 2^{15}$ Rahmen.

b) Adresse $0x4B301568$, Pagegröße 64 KiB: Seitennummer $0x4B30$, Offset $0x1568$. Kleinste Adresse: $0x4B300000$. Größte: $0x4B30FFFF$.

c) 82114 Byte, Startoffset $0xFE01=65025$. $\text{floor}((65025+82114-1)/65536)+1=3$. Also 3 Rahmen.

Seq	4	5	3	5	6	5	2	5	6
OPT R1	4	5	5	5	5	5	5	5	5
OPT R2	3	3	3	3	3	3	2	2	2
OPT R3	6	6	6	6	6	6	6	6	6
F	F	F					F		

Seq	4	5	3	5	6	5	2	5	6
LRU R1	4	4	4	4	6	6	6	6	6
LRU R2	3	5	5	5	5	5	5	5	5
LRU R3	6	6	3	3	3	3	2	2	2

F		F	F	F		F		F		
---	--	---	---	---	--	---	--	---	--	--

Aufgabe 4: Dateisysteme

a) $2^{16} * 512 = 33.554.432 \text{ Byte} = 32 \text{ MiB}$.

b) Clustergröße bei $i=4$: 16 Blöcke = 8192 Byte. Obergrenze: $2^{16} * 8192 = 512 \text{ MiB}$.

c) 652 Byte: ohne Cluster 2 Blöcke = 1024 Byte; mit 8192-Byte-Cluster: 8192 Byte.

d) $(8 + 4 * 128) * 512 = 520 * 512 = 266.240 \text{ Byte}$.

e) Vorteil zusammenhängend: sehr schneller sequentieller und wahlfreier Zugriff. Nachteil: externe Fragmentierung und schwieriges Wachstum. Gut geeignet für überwiegend lesende, selten veränderte Medien/Dateien.

Aufgabe 5: Semaphore

a) Kein Deadlock. P2 kann S2/S3 erst nach Freigabe von S1 anfordern; P1 hält S1 bis nach seinem kritischen Ablauf. Es entsteht keine Situation, in der P1 und P2 gegenseitig je eine benötigte Semaphore halten.

b) Wie bei binären Semaphoren: wait belegt bei freier Semaphore oder blockiert den Prozess; signal gibt frei und weckt ggf. genau einen wartenden Prozess. Operationen sind atomar und werden vom Betriebssystem verwaltet.

Prüfung Betriebssysteme WS 2019/20 - Musterlösung

Aufgabe 1: Synchronisation/Deadlocks

a) Race Condition: Das Ergebnis hängt vom zeitlichen Ablauf nebenläufiger Zugriffe auf gemeinsame Daten ab.

b) Nicht geeignet: Prüfung `while(lock==1)` und Setzen `lock=1` sind nicht atomar. Zwei Threads können beide `lock=0` sehen und beide eintreten.

c) Möglichkeiten: atomare Test-and-Set/CAS-Locks, Mutex/Semaphore, Monitore.

d) Situation P1 bei `wait(S3)`, P2 bei `wait(S1)`: P1 hält S2 und wartet auf S3; P2 hält S3 und wartet auf S1. Da S1 frei ist, ist das noch kein Deadlock.

e) Ja, andere Situation: P1 hält S2 und wartet bei `wait(S3)`; P2 hält S3 und S1 und wartet bei `wait(S2)`. Zyklus: $P1 \rightarrow S3 \rightarrow P2 \rightarrow S2 \rightarrow P1$.

f) Barriere für $n=5$: `mutex=1, barrier=0, count=0`. Thread: `wait(mutex); count++; if count==5 signal(barrier); signal(mutex); wait(barrier); signal(barrier);`. Erst der fünfte Thread öffnet das Drehkreuz.

Aufgabe 2: RMS

a) Notwendig: $U = 3/8 + 2/5 + 1/6 \approx 0,942 \leq 1$. Grundsätzlich CPU-zeitlich möglich.

b) RMS-Prioritäten: $T2(P=5) > T3(P=6) > T1(P=8)$. Zeitdiagramm: T1 0-1, T2 1-3, T1 3-4, T3 4-5, T1 5-6, T2 6-8, T1 8-10, T3 10-11, T2 11-13, T1 13-14, T2 16-18, T3 18-19, T1 19-20. Verdrängungen: $t=1, t=4, t=10$. Keine Deadline-Verletzung bis $t=20$.

c) Hinreichende RMS-Grenze für $n=3$: $3(2^{(1/3)}-1) \approx 0,779$. Da $0,942$ darüber liegt, gibt die einfache RMS-Grenze keine Garantie.

d) Prioritätsinvertierung: hoher Task wartet auf Ressource eines niedrigen Tasks, während mittlere Tasks den niedrigen verdrängen. Dadurch wartet der hohe Task indirekt auf mittlere Tasks.

e) Lösung: Priority Inheritance oder Priority Ceiling; der niedrige Ressourceneigentümer wird temporär auf höhere Priorität angehoben.

Aufgabe 3: Speicher

a) $1 \text{ GiB} / 2 \text{ KiB} = 2^{30} / 2^{11} = 2^{19}$ Rahmen.

b) `0x8B0D09F8`, Seitengröße `0x800`: kleinste Adresse `0x8B0D0800`, größte `0x8B0D0FFF`.

c) 9172 Byte: minimal $\text{ceil}(9172/2048)=5$, maximal 6 Rahmen.

d) Start 0x31BA2FFE, Offset 0x7FE=2046. $\text{floor}((2046+9172-1)/2048)+1 = 6$ Rahmen.

Seq	1	3	4	5	1	5	4	2	1	5
FIFO R1	5	3	3	3	1	1	1	1	1	1
FIFO R2	1	1	4	4	4	4	4	2	2	2
FIFO R3	2	2	2	5	5	5	5	5	5	5
F		F	F	F	F			F		

Seq	1	3	4	5	1	5	4	2	1	5
OPT R1	5	5	5	5	5	5	5	5	5	5
OPT R2	1	1	1	1	1	1	1	1	1	1
OPT R3	2	3	4	4	4	4	4	2	2	2
F		F	F					F		

Aufgabe 4: Dateisysteme

a) System A ohne Tabelle: Nutzdaten pro Block $512-2=510$. $156 \text{ B} \rightarrow 1 \text{ Block} = 512 \text{ B}$. $3065 \text{ B} \rightarrow \text{ceil}(3065/510)=7$ Blöcke = 3584 B. System B/FAT: $156 \text{ B} \rightarrow 512 \text{ B}$; $3065 \text{ B} \rightarrow \text{ceil}(3065/512)=6$ Blöcke = 3072 B.

b) Random Access = direkter Zugriff auf beliebige Dateiposition. FAT ist besser als verkettete Liste ohne Tabelle, weil die Kette zentral in der Tabelle verfolgt werden kann; zusammenhängende oder indexierte Belegung wäre noch besser.

c) Für Auslagerungsdatei eignet sich B besser als A, weil Seiten direkt/regelmäßiger adressiert werden müssen und die Datenblöcke bei A durch Zeigerplatz verkleinert sind.

d) $2^{16} \cdot 512 = 32 \text{ MiB}$. FAT-Größe: $2^{16} \cdot 2 \text{ Byte} = 128 \text{ KiB}$.

e) Zusammenhängend: Vorteil sehr schneller direkter/sequentieller Zugriff; Nachteil externe Fragmentierung und schwieriges Wachstum. Gut bei vielen Lesezugriffen und wenigen Änderungen.

f) Datenblöcke: $8 + 2 \cdot 256 + 2 \cdot 256^2 = 131592$. Bei 512 B: 67.375.104 Byte.

Exam Operating Systems SS 2021 - Sample Solution

Question 1: Synchronization/Deadlocks

a) No deadlock at t_0 . P1 waits for free R2; P2 waits for R3 held by P3; P3 waits for R5 held by P1. Since R2 is free, P1 can proceed and later release R5.

b) P1 must not request R1, R3 or R4. R1/R3 are held by P3 and produce a cycle through $P3 \rightarrow R5 \rightarrow P1$; R4 is held by P2 and produces $P1 \rightarrow R4 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R5 \rightarrow P1$.

c) The resource graph can be used for deadlock avoidance: before granting a request, check whether the new edge would create an unsafe cycle.

d) The attempt is not safe because testing $flag == 1$ and setting $flag = 1$ are not atomic. Two processes may both observe 0, both leave the loop, both set flag to 1, and both enter the critical region.

e) Even if correct, it would be inefficient because the waiting process busy-waits in the while-loop and consumes CPU time.

f) A binary semaphore is kernel-controlled and manipulated by atomic wait/signal operations. Value 1 means free, 0 means occupied. Waiting processes are blocked in a queue, not spinning in user code; user programs cannot freely change the internal variable.

g) Rendezvous with semaphores: initialize $s1=0, s2=0$. P1 before barrier: $signal(s1); wait(s2)$; after barrier. P2 before barrier: $signal(s2); wait(s1)$; after barrier. Each process releases the other only after arriving.

Question 2: RMS

a) Necessary condition: $U \leq 1$. $U = 3/8 + 1/4 + 2/6 = 0.9583 \leq 1$, so CPU time is not impossible.

b) Sufficient RMS bound: $U \leq n(2^{1/n} - 1)$. For $n=3$: about 0.779. Since $0.9583 > 0.779$, the bound is not satisfied, so RMS does not get a simple guarantee.

c) Priorities: $T2(P=4) > T3(P=6) > T1(P=8)$. Schedule:

T1 0-1, T2 1-2, T3 2-4, T1 4-5, T2 5-6, T1 6-7, T3 8-9, T2 9-10, T3 10-11, T1 11-13, T2 13-14, T3 14-16, T1 16-17, T2 17-18, T1 18-20

Preemptions: P at $t=1, 5, 9, 13$. Deadline miss: X at $t=16$ for T1's job released at $t=8$.

d) Relevant real-time goals: meeting deadlines/predictability, bounded response time/low latency, and low jitter. A real-time scheduler is judged by timeliness, not only average throughput.

Question 3: Memory

a) $1 \text{ GiB} / 16 \text{ KiB} = 2^{30} / 2^{14} = 2^{16}$ frames.

b). Page table entries: $2^{32} / 2^{14} = 2^{18}$.

c). Address $0x1B32FFE0$, page size 16 KiB = 14 offset bits. Page number: $0x6CCB$, offset $0x3FE0$.

Seq	3	1	4	5	1	5	4	2	1	5
LRU R1	3	3	3	5	5	5	5	5	1	1
LRU R2	1	1	1	1	1	1	1	2	2	2
LRU R3	2	2	4	4	4	4	4	4	4	5
F	F		F	F				F	F	F

Seq	3	1	4	5	1	5	4	2	1	5
LFU R1	3	3	3	5	5	5	5	5	5	5
LFU R2	1	1	1	1	1	1	1	1	1	1
LFU R3	2	2	4	4	4	4	4	2	2	2
F	F		F	F				F		

f). Temporal locality: recently used data/instructions are likely to be used again. Spatial locality: addresses near a used address are likely to be used soon.

g). Correct: Caching. Locking and spooling do not primarily rely on locality.

Question 4: File Systems

a). Contiguous allocation: $\text{ceil}(100,000,000/1024)=97,657$ blocks, so 100,000,768 bytes.

b). Linked list without table: 32-bit pointer = 4 bytes, useful payload per block = 1020 bytes.
 $\text{ceil}(100,000,000/1020)=98,040$ blocks, so 100,392,960 bytes.

c). I-Node: data blocks 97,657. Pointer blocks needed: 1 single-indirect block + 1 double-indirect root + 381 leaf pointer blocks = 383, plus 1 I-Node block. Total $97,657+383+1=98,041$ blocks = 100,393,984 bytes.

d). Random access fastest to slowest: A, C, B. A needs the start block and direct offset calculation, so roughly one target block. C needs index lookup (I-node and indirect blocks, for the last block here up to I-node + double root + indirect leaf + data block). B must follow the linked list through almost all previous blocks to reach the last block.